Iterative Refinement

Design and Analysis of Algorithms

Brian C. Dean

School of Computing

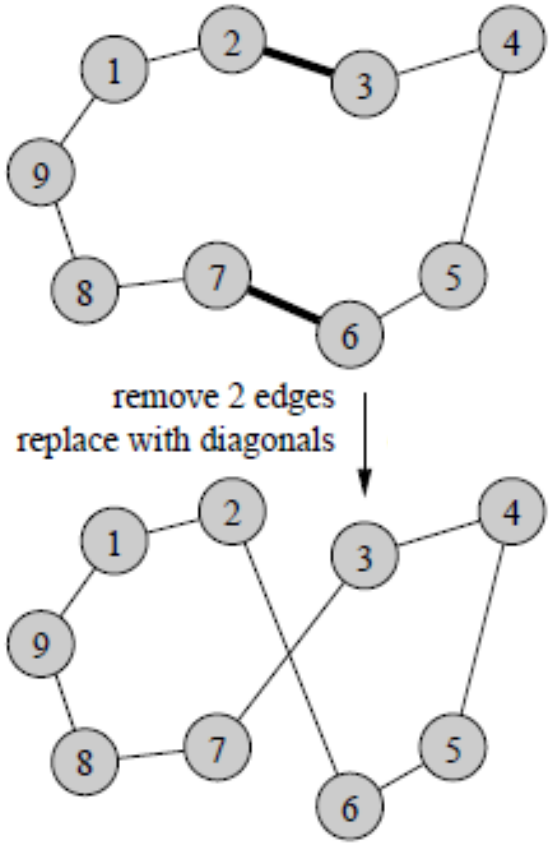
Clemson University

Iterative Refinement

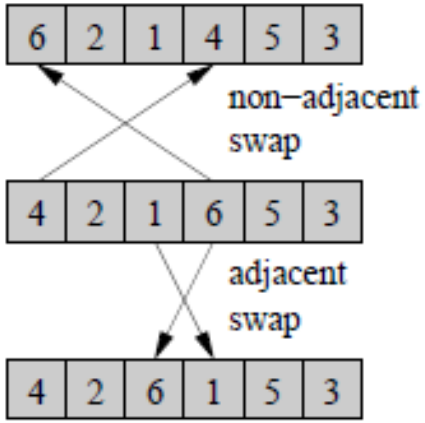
- Simple yet powerful idea:
  - Start with some arbitrary feasible solution
    (or better yet, a solution obtained via some other heuristic)
  - As long as we can improve it, keep making it better (e.g., search a small "neighborhood" of solutions similar to x, moving to better one if found)
  - Simple example: bubble sort.
- Getting stuck in local minima can be a problem.
  - Several ways to mitigate; e.g., run multiple times from different random starting points.

Local Search – Neighborhood Examples

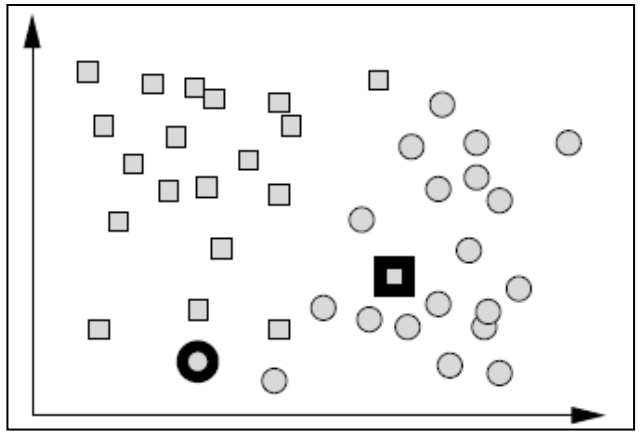
Traveling salesman:



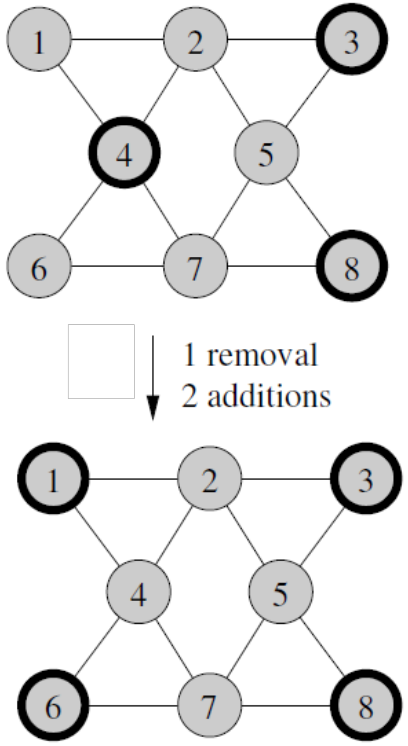
Rank aggregation:



Clustering:



Max independent set:



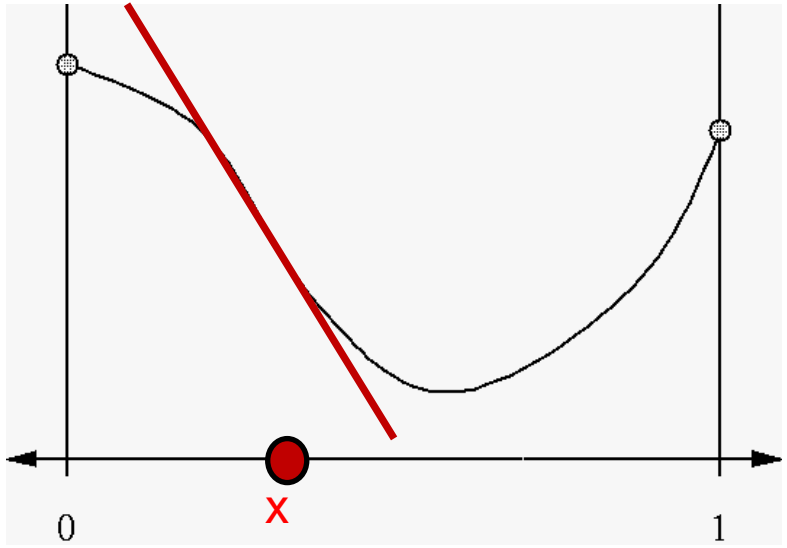
Neighborhood size? (large vs. small)

Movement strategy? (immediate vs. after searching entire neighborhood)

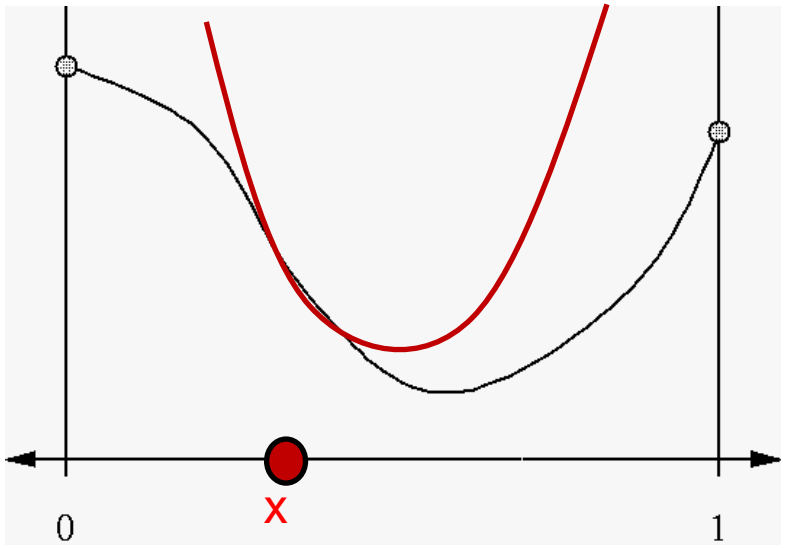
What to do if infeasible solutions are generated? (prune vs. repair vs. penalize)

Continuous Optimization (Discussed Later…)

- To refine a guess x, locally approximate function being minimized with a linear function or a quadratic.



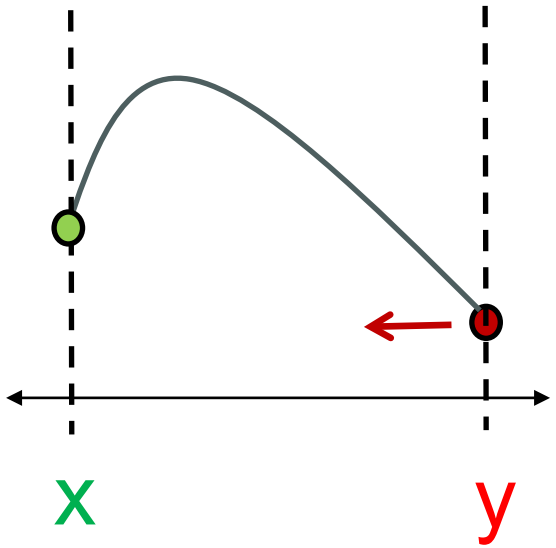
Gradient Descent
(if derivatives easy to compute)



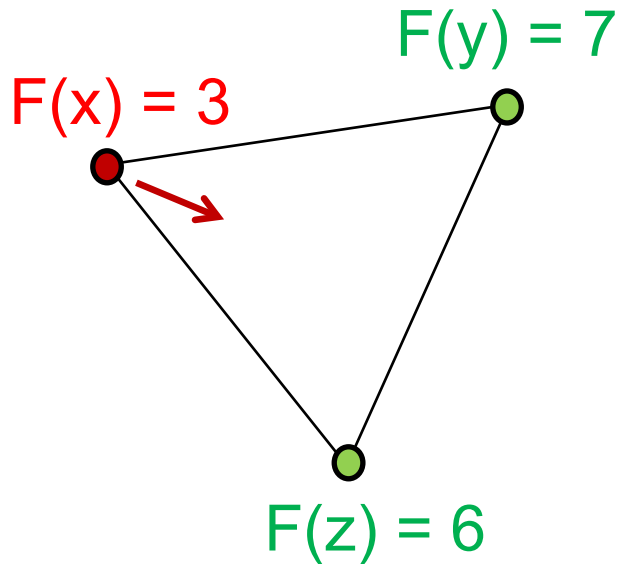
Newton's Method
(if second derivatives easy to compute)

“Binary Search” in Optimization (From Assignment #1 Discussion)

The “Nelder-Mead” / simplex search / amoeba search algorithm:



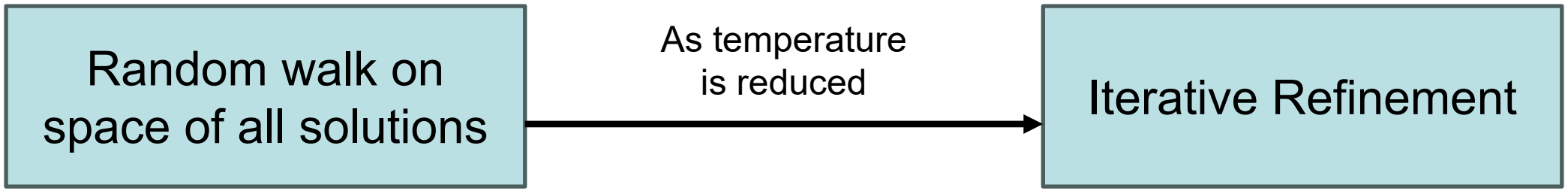
In d dimensions, “bracket” solution with d+1 points



Move “bracket” with worst function value in direction of others

Simulated Annealing

- Applicable for discrete or continuous optimization.
- Generate neighbor x' of current solution x.
  - If x' better, move there.
  - Otherwise, move with probability like $e^{-\Delta/T}$, where Δ measures degradation in objective and T is current "temperature".

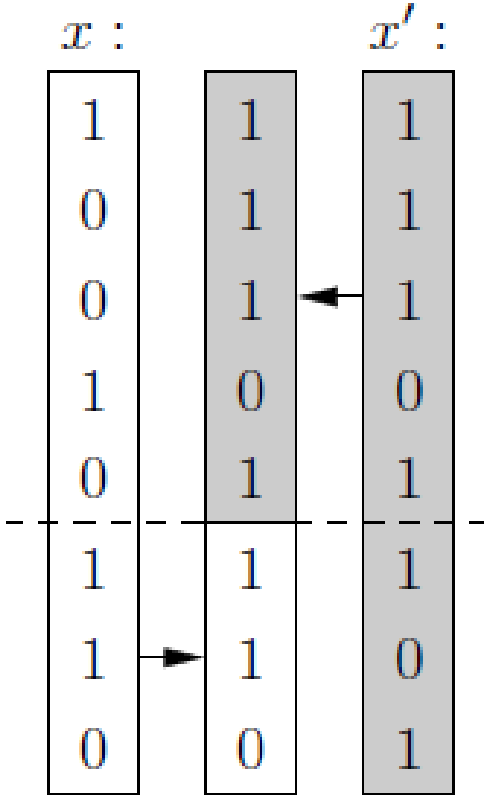


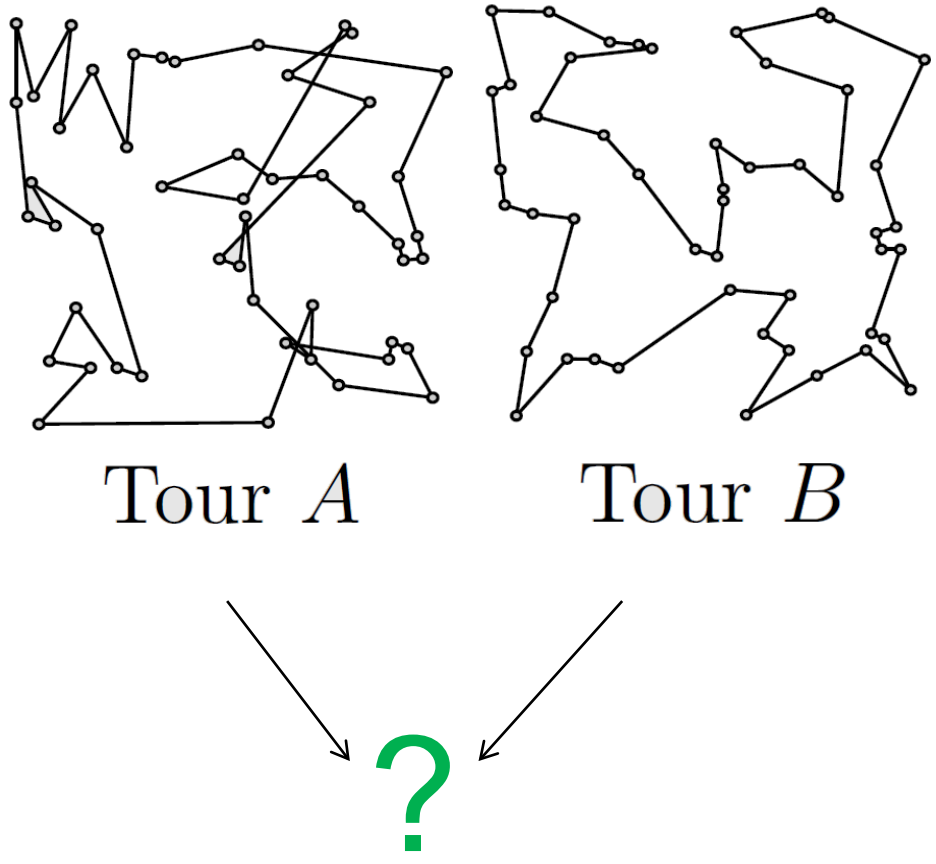
- Can help escape local minima.
- Related to diffusion-based sampling methods in generative AI.

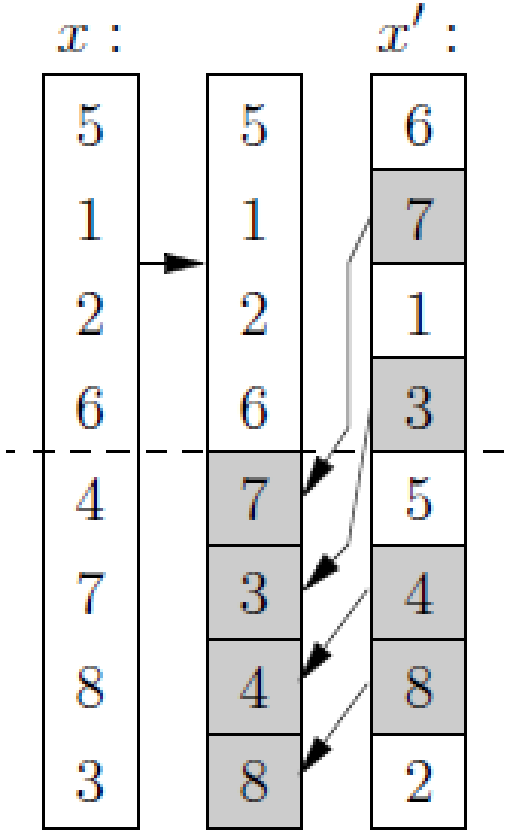
Population-Based Search: Genetic Algorithms and Relatives

- Start with a **population** of initial solutions.
- Each member of next generation obtained by:
  - Mutation of good solution from previous generation,
  - Result of “mating” together two good solutions from previous generation, or
  - Direct copy of best solution(s) from previous generation.
- Solutions from previous generation chosen with probability related to their **fitness** (“roulette wheel” selection).
- Carefully balance mutation / crossover rates to prevent collapse.
- Can generate not one but population of diverse solutions, and also solutions that are robust against mutation / perturbation.

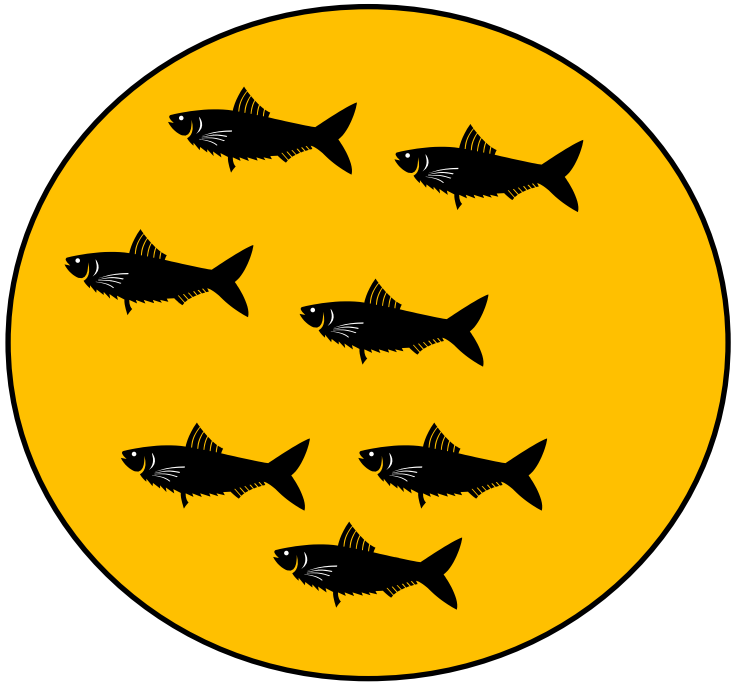
Genetic Algorithms: “Crossover” for Combining Solutions…







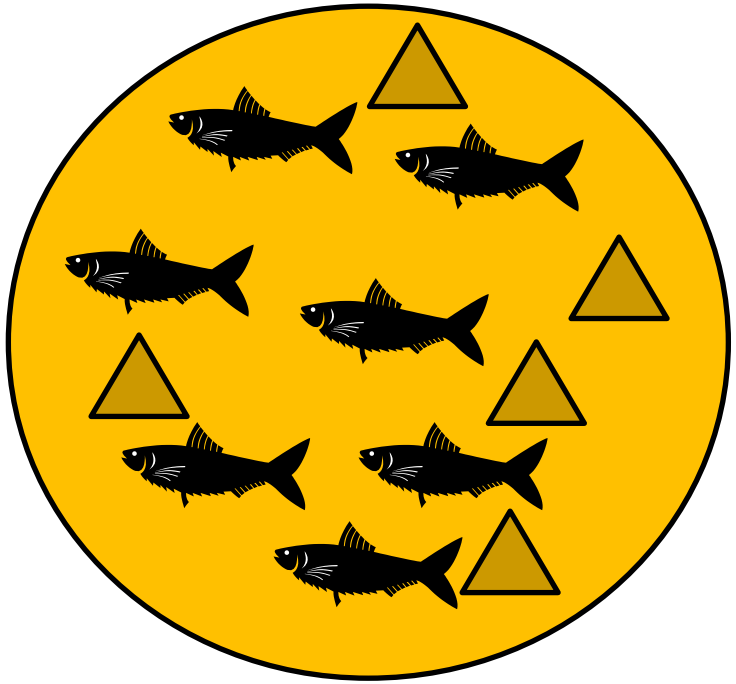
Crossover Example



Sardine Cookies

Liver and Chocolate Chips

Crossover Example



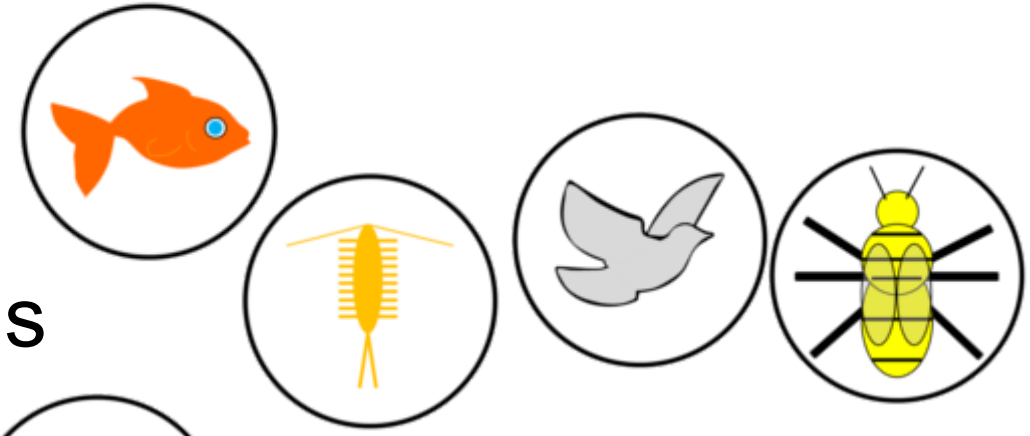
Liver & Sardine Cookies!

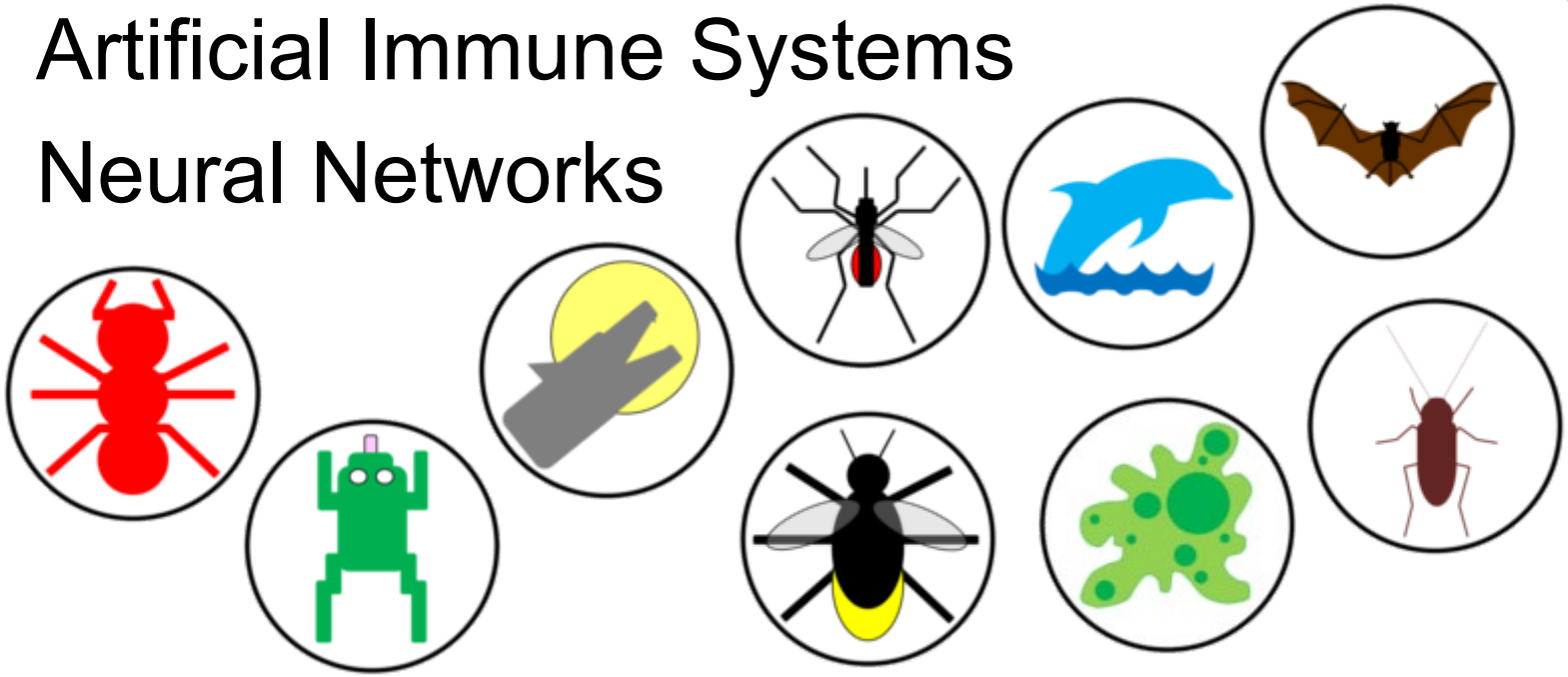
Crossover Example

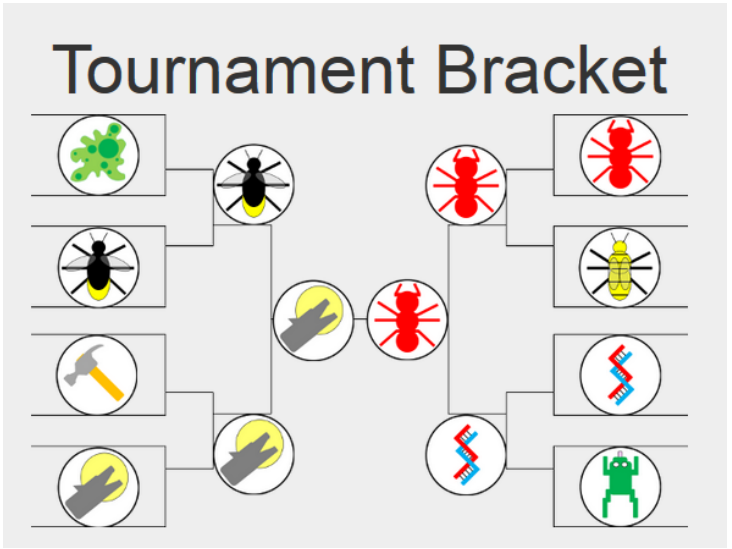
Liver & Sardine Cookies!
(Don’t ever make these)

Physical and Biologically Inspired Computing Paradigms

- Simulated Annealing
- Genetic/Evolutionary Algorithms
- Bio-Inspired Optimization Methods
- Artificial Immune Systems
- Neural Networks







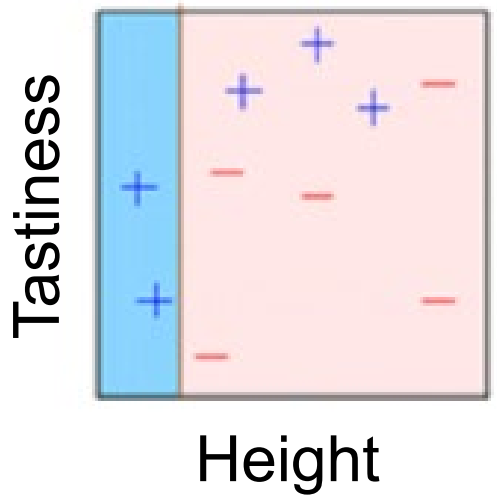
Population-Based Refinement and "Social Intelligence"

- The potential advantage of population-based methods over refining just a single solution.
- Attractive beacons in search space: ant colony optimization
- Repulsive beacons: tabu search, solutions maintaining "personal space"
- Related: balancing exploration vs. exploitation
- "Swarm intelligence": using group characteristics (centroid, average velocity) in solution updates
- Colonies of solutions with periodic migration.

Boosting: Refine by Adding to Solution

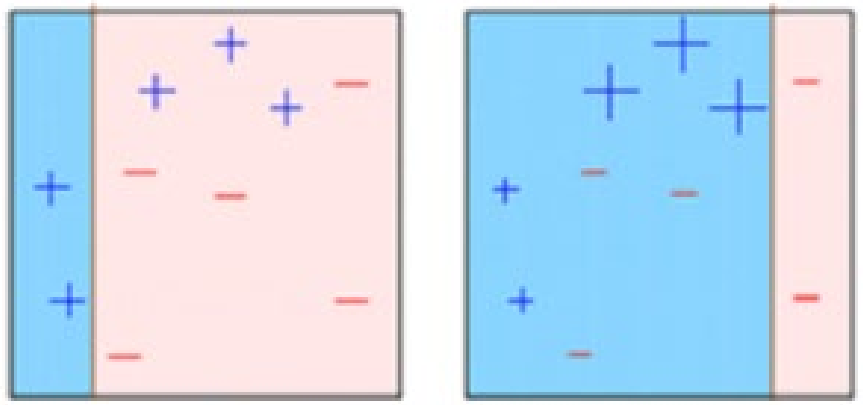
- Boosting and “ensemble” classification… turns “weak learners” into “strong learners”.
- Suppose you can guarantee building a classifier that has only 51% accuracy (a “weak” learner)…

- E.g., should this object be classified as a pancake?



Boosting

- Boosting and “ensemble” classification… turns “weak learners” into “strong learners”.
- Suppose you can guarantee building a classifier that has only 51% accuracy (a “weak” learner)…
- Boost “weight” of misclassified samples and find another classifier, building an ensemble of weak classifiers that classifies by weighted majority vote.

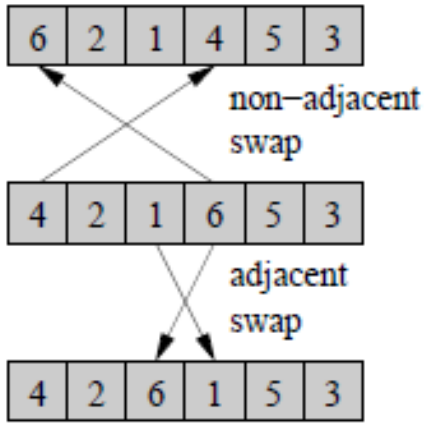


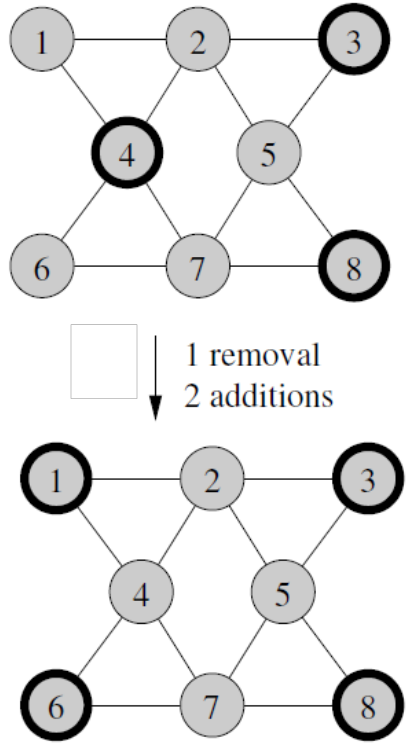
- Error rate of ensemble drops exponentially with # of iterations!

Refine by Averaging Weak Results Over a Series of Iterations

- Bagging: solve problem on independent subsamples of input, average results together.
  - E.g., train on samples of training data and/or features
- Average solutions generated by simple heuristics using random starting points – resulting "heat map" can inform how to build final solution.

Rank aggregation:





Max independent set

Multi-Scale Methods